```
import React, { useState, useEffect } from 'react';
import {
  ChefHat,
  BookOpen,
  User,
  Flame,
  Sparkles,
  Camera,
  Loader2,
  Lock,
  Star,
  CheckCircle,
  ArrowRight,
  LogOut,
} from 'lucide-react';

export default function App() {
  // Authentication State
  const [isAuthenticated, setIsAuthenticated] = useState(false);

  // App States
  const [activeTab, setActiveTab] = useState('learn');
  const [xp, setXp] = useState(1250);
  const [streak, setStreak] = useState(12);
  const [isPremium, setIsPremium] = useState(false);

  // If not logged in, show the Auth Screen
  if (!isAuthenticated) {
    return <AuthScreen onLogin={() => setIsAuthenticated(true)} />;
  }

  return (
    <div className="flex flex-col h-screen max-w-md mx-auto bg-white font-sans text-black border-
x-2 border-black shadow-2xl relative overflow-hidden">
      {/* Top Header */}
      <header className="flex justify-between items-center p-4 bg-white border-b-2 border-black z-10
shrink-0">
        <div className="flex items-center space-x-2">
          <ChefHat className="w-8 h-8 text-black" />
          <h1 className="text-2xl font-black tracking-tight text-black">Cookify</h1>
          {isPremium && (
            <span className="bg-black text-white text-[10px] font-bold px-2 py-1 rounded-md uppercase
tracking-wider">
              Pro
            </span>
```

```
        )}
      </div>
      <div className="flex space-x-4">
        <div className="flex items-center text-gray-500 font-bold">
          <Flame className="w-5 h-5 mr-1 text-black" fill="currentColor" />
          {streak}
        </div>
        <div className="flex items-center text-black font-bold">{xp} XP</div>
      </div>
    </header>

    {/* Main Content Area */}
    <main className="flex-1 overflow-hidden relative bg-gray-50">
      {activeTab === 'learn' && <LearnView />}
      {activeTab === 'ai' && <AIChefView isPremium={isPremium} setIsPremium={setIsPremium} />
}
      {activeTab === 'profile' && (
        <ProfileView
          xp={xp}
          streak={streak}
          isPremium={isPremium}
          setIsPremium={setIsPremium}
          onLogout={() => setIsAuthenticated(false)}
        />
      )}
    </main>

    {/* Bottom Navigation */}
    <nav className="flex justify-around items-center p-3 bg-white border-t-2 border-black pb-safe
shrink-0 z-10">
      <NavButton icon={<BookOpen className="w-6 h-6" />} label="Learn" isActive={activeTab
=== 'learn'} onClick={() => setActiveTab('learn')} />
      <NavButton icon={<Sparkles className="w-6 h-6" />} label="AI Chef" isActive={activeTab
=== 'ai'} onClick={() => setActiveTab('ai')} />
      <NavButton icon={<User className="w-6 h-6" />} label="Profile" isActive={activeTab ===
'profile'} onClick={() => setActiveTab('profile')} />
    </nav>
  </div>
 );
}

// --- AUTHENTICATION SCREEN ---
function AuthScreen({ onLogin }) {
 const [loadingProvider, setLoadingProvider] = useState(null);
```

```
  const handleLogin = (provider) => {
    setLoadingProvider(provider);
    // Simulate network request for OAuth
    setTimeout(() => {
      setLoadingProvider(null);
      onLogin();
    }, 1500);
  };

  return (
    <div className="flex flex-col h-screen max-w-md mx-auto bg-white font-sans text-black border-x-2 border-black shadow-2xl relative overflow-hidden justify-between p-6">
      {/* Decorative Background Elements */}
      <div className="absolute top-[-10%] left-[-10%] w-64 h-64 border-[10px] border-gray-100 rounded-full -z-10" />
      <div className="absolute bottom-[20%] right-[-20%] w-80 h-80 border-[10px] border-gray-100 rounded-full -z-10" />

      {/* Logo & Branding */}
      <div className="flex flex-col items-center justify-center flex-1 mt-12">
        <div className="bg-black text-white p-6 rounded-[2rem] border-4 border-gray-200 shadow-[8px_8px_0px_0px_rgba(0,0,0,1)] mb-8 transform -rotate-3">
          <ChefHat className="w-20 h-20" />
        </div>
        <h1 className="text-5xl font-black tracking-tighter uppercase mb-4 text-center">Cookify</h1>
        <p className="text-gray-500 font-bold text-center uppercase tracking-widest text-sm max-w-[250px] leading-relaxed">
          Master global cuisines. One bite at a time.
        </p>
      </div>

      {/* Login Buttons */}
      <div className="space-y-4 mb-8">
        <button
          onClick={() => handleLogin('apple')}
          disabled={loadingProvider !== null}
          className="w-full bg-white text-black font-black py-4 rounded-2xl border-2 border-black border-b-[6px] active:border-b-2 active:translate-y-[4px] transition-all disabled:opacity-50 flex items-center justify-center uppercase tracking-wide text-lg shadow-sm"
        >
          {loadingProvider === 'apple' ? (
            <Loader2 className="animate-spin w-6 h-6" />
          ) : (
            <>
              <svg className="w-6 h-6 mr-3" viewBox="0 0 384 512" fill="currentColor">
```

```
              <path d="M318.7 268.7c-.2-36.7 16.4-64.4
50-84.8-18.8-26.9-47.2-41.7-84.7-44.6-35.5-2.8-74.3 20.7-88.5 20.7-15 0-49.4-19.7-76.4-19.7C63.3 141.2
4 184.8 4 273.5q0 39.3 14.4 81.2c12.8 36.7 59 126.7 107.2 125.2 25.2-.6 43-17.9 75.8-17.9 31.8 0 48.3
17.9 76.4 17.9 48.6-.7 90.4-82.5 102.6-119.3-65.2-30.7-61.7-90-61.7-91.9zm-56.6-164.2c27.3-32.4
24.8-61.9 24-72.5-24.1 1.4-52 16.4-67.9 34.9-17.5 19.8-27.8 44.3-25.6 71.9 26.1 2 49.9-11.4 69.5-34.3z" /
>
            </svg>
            Continue with Apple
          </>
        )}
      </button>

      <button
        onClick={() => handleLogin('google')}
        disabled={loadingProvider !== null}
        className="w-full bg-white text-black font-black py-4 rounded-2xl border-2 border-black
border-b-[6px] active:border-b-2 active:translate-y-[4px] transition-all disabled:opacity-50 flex items-
center justify-center uppercase tracking-wide text-lg shadow-sm"
      >
        {loadingProvider === 'google' ? (
          <Loader2 className="animate-spin w-6 h-6" />
        ) : (
          <>
            <svg className="w-6 h-6 mr-3" viewBox="0 0 488 512" fill="currentColor">
              <path d="M488 261.8C488 403.3 391.1 504 248 504 110.8 504 0 393.2 0 256S110.8 8 248
8c66.8 0 123 24.5 166.3 64.9l-67.5 64.9C258.5 52.6 94.3 116.6 94.3 256c0 86.5 69.1 156.6 153.7 156.6
98.2 0 135-70.4 140.8-106.9H248v-85.3h236.1c2.3 12.7 3.9 24.9 3.9 41.4z" />
            </svg>
            Continue with Google
          </>
        )}
      </button>

      <button
        onClick={() => handleLogin('guest')}
        disabled={loadingProvider !== null}
        className="w-full bg-gray-100 text-gray-500 font-bold py-4 rounded-2xl border-2 border-
transparent hover:bg-gray-200 transition-all disabled:opacity-50 flex items-center justify-center
uppercase tracking-wide text-sm"
      >
        Skip for now (Guest)
      </button>
    </div>

    <p className="text-[10px] text-center text-gray-400 font-bold uppercase tracking-widest px-4">
```

```
        By continuing, you agree to Cookify's <br />
        <span className="text-black underline">Terms of Service</span> & <span className="text-black underline">Privacy Policy</span>
      </p>
    </div>
  );
}

// --- NAVIGATION COMPONENT ---
function NavButton({ icon, label, isActive, onClick }) {
  return (
    <button
      onClick={onClick}
      className={`flex flex-col items-center p-2 rounded-2xl transition-all ${
        isActive ? 'text-black bg-gray-100 scale-110' : 'text-gray-400 hover:bg-gray-50 hover:text-black'
      }`}
    >
      <div className={`${isActive ? 'opacity-100' : 'opacity-70'} mb-1`}>{icon}</div>
      <span className={`text-[10px] font-bold uppercase tracking-wide ${isActive ? 'opacity-100' : 'opacity-70'}`}>{label}</span>
    </button>
  );
}

// --- LEARN VIEW ---
function LearnView() {
  const levels = [
    { id: 1, title: 'Egg Basics', status: 'completed', icon: <CheckCircle className="text-white w-8 h-8" /> },
    { id: 2, title: 'Knife Skills', status: 'completed', icon: <CheckCircle className="text-white w-8 h-8" /> },
    { id: 3, title: 'Italian Pasta', status: 'active', icon: <ChefHat className="text-black w-8 h-8" /> },
    { id: 4, title: 'French Sauces', status: 'locked', icon: <Lock className="text-gray-400 w-8 h-8" /> },
    { id: 5, title: 'Wok Master', status: 'locked', icon: <Lock className="text-gray-400 w-8 h-8" /> },
  ];

  return (
    <div className="h-full overflow-y-auto p-6 pb-24 scroll-smooth">
      <div className="flex flex-col items-center space-y-8 mt-4">
        {levels.map((level, index) => {
          const isOffset = index % 2 !== 0;
          let btnClass =
            'w-20 h-20 rounded-full flex items-center justify-center border-4 border-black border-b-[8px] active:border-b-4 active:translate-y-1 transition-all z-10 ';
```

```
        if (level.status === 'completed') {
          btnClass += 'bg-black text-white hover:bg-gray-800';
        } else if (level.status === 'active') {
          btnClass += 'bg-white text-black border-black border-b-[8px] animate-pulse';
        } else {
          btnClass += 'bg-gray-100 border-gray-300 text-gray-400 border-b-[8px] border-b-gray-400
cursor-not-allowed';
        }

        return (
          <div key={level.id} className={`flex flex-col items-center relative ${isOffset ? 'ml-24' :
'mr-24'}`}>
            {index < levels.length - 1 && (
              <div
                className={`absolute w-3 h-20 -bottom-16 -z-10 ${isOffset ? '-rotate-[30deg] -left-4' :
'rotate-[30deg] -right-4'} ${
                  level.status === 'locked' ? 'bg-gray-200' : 'bg-black'
                }`}
              />
            )}
            <button className={btnClass} disabled={level.status === 'locked'}>
              {level.icon}
            </button>
            <span className={`mt-3 font-bold uppercase tracking-wide text-xs ${level.status ===
'locked' ? 'text-gray-400' : 'text-black'}`}>
              {level.title}
            </span>
          </div>
        );
      })}
    </div>
  </div>
 );
}

// --- GEMINI API HELPER ---
async function callGeminiApi(payload) {
 const apiKey = ''; // API Key handled by environment / platform
 const model = 'gemini-2.5-flash-preview-09-2025';
 const url = `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?
key=${apiKey}`;
 const delays = [1000, 2000, 4000, 8000, 16000];

 for (let i = 0; i < 5; i++) {
   try {
```

```
      const response = await fetch(url, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload),
      });
      if (!response.ok) throw new Error(`HTTP error ${response.status}`);
      return await response.json();
    } catch (error) {
      if (i === 4) throw error;
      await new Promise((r) => setTimeout(r, delays[i]));
    }
  }
}

// --- AI CHEF VIEW (WITH PREMIUM PAYWALL) ---
function AIChefView({ isPremium, setIsPremium }) {
  const [activeTool, setActiveTool] = useState('recipe');

  return (
    <div className="h-full flex flex-col bg-white">
      <div className="p-4 bg-white border-b-2 border-black shrink-0">
        <h2 className="text-2xl font-black text-black mb-4 flex items-center">
          <Sparkles className="mr-2" /> AI Kitchen
        </h2>
        <div className="flex bg-gray-100 p-1 rounded-2xl overflow-x-auto hide-scrollbar space-x-1
border-2 border-black">
          <button
            onClick={() => setActiveTool('recipe')}
            className={`flex-1 min-w-[100px] py-2 px-1 text-sm font-bold rounded-xl transition-all
border-2 ${
              activeTool === 'recipe' ? 'bg-black text-white border-black' : 'border-transparent text-gray-500
hover:text-black'
            }`}
          >
            Pantry
          </button>
          <button
            onClick={() => setActiveTool('remix')}
            className={`flex-1 min-w-[100px] py-2 px-1 text-sm font-bold rounded-xl transition-all
border-2 flex justify-center items-center ${
              activeTool === 'remix' ? 'bg-black text-white border-black' : 'border-transparent text-gray-500
hover:text-black'
            }`}
          >
            Remixer {!isPremium && <Lock className="w-3 h-3 ml-1" />}
```

```
          </button>
          <button
            onClick={() => setActiveTool('calories')}
            className={`flex-1 min-w-[100px] py-2 px-1 text-sm font-bold rounded-xl transition-all
border-2 flex justify-center items-center ${
              activeTool === 'calories' ? 'bg-black text-white border-black' : 'border-transparent text-
gray-500 hover:text-black'
            }`}
          >
            Calories {!isPremium && <Lock className="w-3 h-3 ml-1" />}
          </button>
        </div>
      </div>

      <div className="flex-1 overflow-y-auto p-4 pb-20 bg-gray-50">
        {activeTool === 'recipe' && <RecipeGenerator />}
        {activeTool === 'remix' && (isPremium ? <FlavorRemixer /> : <PremiumUpsell onUpgrade={()
=> setIsPremium(true)} feature="Flavor Remixer" />)}
        {activeTool === 'calories' && (isPremium ? <CalorieScanner /> : <PremiumUpsell
onUpgrade={() => setIsPremium(true)} feature="Calorie Scanner" />)}
      </div>
    </div>
  );
}

// --- FREE FEATURE ---
function RecipeGenerator() {
  const [ingredients, setIngredients] = useState('');
  const [loading, setLoading] = useState(false);
  const [recipe, setRecipe] = useState(null);

  const generateRecipe = async () => {
    if (!ingredients.trim()) return;
    setLoading(true);
    setRecipe(null);

    const prompt = `You are an expert, encouraging AI chef for a gamified cooking app called
"Cookify".
The user has the following ingredients: ${ingredients}.
Create a simple, delicious recipe they can make using mostly these (and basic pantry staples).
Format the response clearly using Markdown (headers starting with ##, bullet points, numbered
lists).
Keep it concise and minimalist, fitting a black and white app theme.`;

    try {
```

```
      const result = await callGeminiApi({
        contents: [{ parts: [{ text: prompt }] }],
      });
      if (result?.candidates?.[0]?.content) {
        setRecipe(result.candidates[0].content.parts[0].text || String(result.candidates[0].content));
      } else {
        setRecipe("Sorry, Chef! I couldn't come up with a recipe this time. Try adding more ingredients.");
      }
    } catch (error) {
      console.error(error);
      setRecipe('System Error. Please check your connection and try again.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="space-y-4">
      <div className="bg-white p-5 rounded-3xl border-2 border-black shadow-[4px_4px_0px_0px_rgba(0,0,0,1)]">
        <label className="block font-black text-black mb-2 uppercase text-sm tracking-wider">What's in your fridge?</label>
        <textarea
          className="w-full bg-white border-2 border-gray-300 rounded-2xl p-3 focus:outline-none focus:border-black focus:ring-1 focus:ring-black transition-colors resize-none text-black"
          rows="3"
          placeholder="e.g., chicken breast, rice, broccoli..."
          value={ingredients}
          onChange={(e) => setIngredients(e.target.value)}
        />
        <button
          onClick={generateRecipe}
          disabled={loading || !ingredients.trim()}
          className="w-full mt-4 bg-black text-white font-black py-3 rounded-2xl border-b-[6px] border-gray-800 active:border-b-0 active:translate-y-[6px] transition-all disabled:opacity-50 flex items-center justify-center uppercase tracking-wide"
        >
          {loading ? <Loader2 className="animate-spin mr-2 w-5 h-5" /> : <ChefHat className="mr-2 w-5 h-5" />}
          {loading ? 'Cooking...' : 'Generate Recipe'}
        </button>
      </div>

      {recipe && (
```

```
      <div className="bg-white p-5 rounded-3xl border-2 border-black shadow-[4px_4px_0px_0px_rgba(0,0,0,1)] animate-fade-in-up">
        <div className="prose prose-p:text-black prose-headings:text-black max-w-none text-black text-sm leading-relaxed whitespace-pre-wrap">
          {recipe.split('\n').map((line, i) => {
            if (line.startsWith('## ')) return <h2 key={i} className="text-xl font-black mt-4 mb-2 uppercase border-b-2 border-black pb-1 inline-block">{line.replace('## ', '')}</h2>;
            if (line.startsWith('# ')) return <h1 key={i} className="text-2xl font-black mb-3 uppercase">{line.replace('# ', '')}</h1>;
            if (line.startsWith('* ')) return <p key={i} className="mb-2 font-medium">• {line.replace('* ', '')}</p>;
            return <p key={i} className="mb-2 font-medium" dangerouslySetInnerHTML={{ __html: line.replace(/\*\*(.*?)\*\*/g, '<strong>$1</strong>') }} />;
          })}
        </div>
      </div>
    )}
  </div>
 );
}

// --- PREMIUM UPSELL COMPONENT ---
function PremiumUpsell({ onUpgrade, feature }) {
 return (
  <div className="bg-black text-white p-6 rounded-3xl border-4 border-gray-200 text-center relative overflow-hidden mt-4 shadow-2xl">
    <div className="absolute top-0 right-0 -mr-8 -mt-8 text-gray-800 opacity-20">
      <Star className="w-48 h-48" fill="currentColor" />
    </div>

    <div className="relative z-10 flex flex-col items-center">
      <div className="bg-white text-black p-3 rounded-full mb-4">
        <Lock className="w-8 h-8" />
      </div>
      <h3 className="text-2xl font-black mb-2 uppercase tracking-wide">Cookify Pro</h3>
      <p className="text-gray-300 font-medium mb-6 text-sm">
        Unlock the <strong className="text-white">{feature}</strong> and take your culinary skills to the next level with our advanced AI tools.
      </p>

      <ul className="text-left space-y-3 mb-8 w-full max-w-xs mx-auto">
        <li className="flex items-center text-sm font-bold">
          <CheckCircle className="w-5 h-5 mr-3 text-white" /> Unlimited Recipes
        </li>
        <li className="flex items-center text-sm font-bold">
```

```
          <CheckCircle className="w-5 h-5 mr-3 text-white" /> Flavor Remixer Tool
        </li>
        <li className="flex items-center text-sm font-bold">
          <CheckCircle className="w-5 h-5 mr-3 text-white" /> Photo Calorie Scanner
        </li>
      </ul>

      <button
        onClick={onUpgrade}
        className="w-full bg-white text-black font-black py-4 rounded-2xl border-b-[6px] border-gray-400 active:border-b-0 active:translate-y-[6px] transition-all flex items-center justify-center uppercase tracking-wider text-lg"
      >
        Upgrade Now <ArrowRight className="ml-2 w-5 h-5" />
      </button>
    </div>
  </div>
 );
}

// --- PREMIUM FEATURE: FLAVOR REMIXER ---
function FlavorRemixer() {
 const [dish, setDish] = useState('');
 const [cuisine, setCuisine] = useState('');
 const [loading, setLoading] = useState(false);
 const [recipe, setRecipe] = useState(null);

 const generateRemix = async () => {
  if (!dish.trim() || !cuisine.trim()) return;
  setLoading(true);
  setRecipe(null);

  const prompt = `You are an expert culinary AI for the "Cookify" app.
The user wants to take a classic dish: "${dish}" and remix it using the flavor profile of "${cuisine}" cuisine.
Provide a creative, delicious recipe for this fusion dish.
Format with Markdown. Include a catchy fusion name, prep/cook time, ingredients list, step-by-step instructions. Keep formatting minimalist.`;

  try {
   const result = await callGeminiApi({
    contents: [{ parts: [{ text: prompt }] }],
   });
   if (result?.candidates?.[0]?.content) {
    setRecipe(result.candidates[0].content.parts[0].text || String(result.candidates[0].content));
```

```
  } else {
    setRecipe('System could not compute fusion. Try different inputs.');
  }
} catch (error) {
  console.error(error);
  setRecipe('System Error. Please check your connection and try again.');
```